THE PREMIUM GUIDE

BACKEND INTERVIEW QUESTIONS

13 topics. 130 questions. The exact questions asked in backend interviews at product companies — with what interviewers are actually testing for in each answer.

TOPICS COVERED

01	Database Indexing	●●●●●
02	Transactions & Isolation Levels	●●●●●
03	Concurrency	●●●●□
04	Caching	●●●●●
05	Idempotency	●●●●□
06	Message Queues	●●●●□
07	Distributed Systems	●●●●□
08	API Design	●●●●●
09	Database Design	●●●●□
10	Authentication & Authorization	●●●●□
11	Docker & Deployment	●●●□□
12	Error Handling & Logging	●●●□□
13	Production Debugging	●●●●●

HOW EACH PAGE IS STRUCTURED

Every topic gets one page: what the concept is, real interview questions asked frequently, and — most importantly — what the interviewer is actually testing for in each answer. Not just Q&A. The reasoning behind it.

TOPIC 01

ASKED EVERYWHERE

Database Indexing

Indexing is the #1 performance topic in backend interviews. Interviewers don't want definitions — they want to know if you've actually run EXPLAIN, seen a slow query, and fixed it.

01 What is a B-tree index and how does it work internally?

▸ They want to know if you understand it's a sorted data structure, not just 'makes queries faster'.

02 What is the difference between a clustered and non-clustered index?

▸ Clustered = data rows stored in index order (one per table). Non-clustered = pointer to the row. Common confusion point.

03 What is a composite index? Explain the left-prefix rule.

▸ An index on (A, B, C) only helps queries filtering on A, or A+B, or A+B+C. Not B alone. This trips people up constantly.

04 When would you NOT add an index?

▸ High write volume tables, low cardinality columns (boolean, status with 3 values), small tables. They test if you know the write cost.

05 What is a covering index?

▸ An index that contains all columns a query needs — so the DB never touches the actual table. Key for read-heavy APIs.

06 How do you diagnose a slow query in production?

▸ EXPLAIN / EXPLAIN ANALYZE. Look for Seq Scan on large tables, high row estimates, missing index. Then verify with actual query time.

07 What is index cardinality and why does it matter for query planning?

▸ High cardinality (unique values) = index is useful. Low cardinality (3 values) = full table scan may be faster. Query planner uses stats.

08 How does an index affect INSERT, UPDATE, and DELETE performance?

▸ Every write must also update every index. 10 indexes = 10x write overhead. Interviewers test if you understand this trade-off.

09 What is a hash index and when is it better than a B-tree?

▸ O(1) lookup for exact match equality, but no range queries or sorting. Use for exact-match lookups only. B-tree wins whenever you need ranges or ordering.

10 How do you decide which columns to index in a large table?

▸ Look at actual query patterns (WHERE, JOIN, ORDER BY columns), not guesses. Index high-selectivity filter columns. Check the slow query log first — don't index speculatively.

TOPIC 02

ASKED EVERYWHERE

Transactions & Isolation Levels

This is one of the most misunderstood backend topics. Most developers use transactions but can't explain what isolation level their app runs at or what bugs that might cause.

01 What are ACID properties? Explain each one with a real example.

▸ Atomicity: all or nothing. Consistency: valid state to valid state. Isolation: concurrent transactions don't see each other's work. Durability: committed = persisted.

02 What are the 4 isolation levels? What problem does each one prevent?

▸ READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE. Each prevents a class of anomaly. Most DBs default to READ COMMITTED.

03 What is a dirty read, phantom read, and non-repeatable read?

▸ Dirty: reading uncommitted data. Non-repeatable: same query returns different values in same transaction. Phantom: new rows appear between two reads.

04 What is a deadlock? How do you detect and prevent one?

▸ Two transactions each waiting for the other's lock. Detection: DB kills one. Prevention: consistent lock ordering, timeouts, keep transactions short.

05 What is the difference between optimistic and pessimistic locking?

▸ Pessimistic: lock the row before reading. Optimistic: read freely, check version on write, retry if changed. Optimistic scales better under low contention.

06 When would you use SERIALIZABLE isolation and what does it cost you?

▸ When you can't tolerate any anomalies (financial systems). Cost: serialization failures, retries, dramatically lower throughput. Rarely the right default.

07 What is a two-phase commit and what problem does it solve?

▸ Coordinates a transaction across multiple databases: prepare phase (all agree), commit phase (all execute). Solves distributed atomicity. High latency cost.

08 What is a savepoint? When would you use one?

▸ A rollback point within a transaction. Roll back part of a transaction without aborting the whole thing. Useful for nested operations with partial failure handling.

09 What is a lost update and which isolation level prevents it?

▸ Two transactions read the same row, both write back based on stale data — one update overwrites the other. REPEATABLE READ or higher (or optimistic locking) prevents it.

10 How do long-running transactions cause problems in production?

▸ They hold locks and block cleanup processes, bloating tables. They increase deadlock risk and block other writers. Keep transactions as short as possible.

TOPIC 03

VERY COMMON

Concurrency

Concurrency bugs are the hardest to reproduce and the most expensive in production. Interviewers use this topic to test if you understand what actually happens when two requests hit the same code at the same millisecond.

01 What is a race condition? Give a real backend example.

▸ Two requests read a balance (100), both deduct 50, both write 50 back. Result: 50 not 0. Classic lost update. Shows up in payments, inventory, counters.

02 What is the difference between concurrency and parallelism?

▸ Concurrency: dealing with multiple things at once (interleaved, one CPU). Parallelism: doing multiple things at once (multiple CPUs). Node.js is concurrent, not parallel.

03 How does Node.js handle concurrent requests without multiple threads?

▸ Event loop + non-blocking I/O. One thread processes callbacks. I/O operations are handed to the OS and a callback fires when done. CPU-bound work blocks the loop.

04 What is an atomic operation? Why does it matter for counters?

▸ Executes as a single indivisible unit. Redis INCR is atomic. SQL UPDATE counter = counter + 1 is atomic. Reading then writing is not. Interviewers test this gap.

05 What is a connection pool and what happens if you don't use one?

▸ Reuses DB connections instead of opening a new one per request. Without it: connection overhead per request, DB runs out of connections under load. Always use one.

06 What is a distributed lock and when do you need one?

▸ A lock across multiple service instances. Use when multiple instances might process the same job or modify the same resource. Redis SETNX or Redlock algorithm.

07 What is the thundering herd problem in the context of concurrency?

▸ Many threads/processes all wake up at once (cache expiry, lock release) and slam a resource. Solution: jitter, probabilistic early expiry, single lock for cache refresh.

08 How do you make a counter increment safe under high concurrency?

▸ Database atomic update (UPDATE SET count = count + 1), Redis INCR, or a dedicated counter service. Never: read, increment in app, write back.

09 What is a mutex vs a semaphore?

▸ Mutex: binary lock, one owner, used for mutual exclusion. Semaphore: counting lock, allows N concurrent holders. Interviewers check if you know mutex isn't just a fancier lock.

10 How would you rate-limit an API safely under concurrent requests?

▸ Use an atomic counter in Redis (INCR + EXPIRE) or a token bucket algorithm, never in-memory counters per instance — that breaks under multiple servers.

TOPIC 04

ASKED EVERYWHERE

Caching

Caching questions are in almost every senior backend interview. The wrong answers are 'use Redis' and 'add a TTL'. The right answers involve strategy, invalidation, and failure modes.

01 Explain the four main caching strategies and when to use each.

▸ Cache-aside (app manages), write-through (write to cache + DB together), write-behind (write to cache, async to DB), refresh-ahead (preload before expiry). Each has a failure mode.

02 What is cache invalidation and why is it considered one of the hardest problems in CS?

▸ Knowing when to remove or update a cached value. Invalidate too early = cache miss storm. Too late = stale data. No universal solution — strategy depends on staleness tolerance.

03 What is a cache stampede / thundering herd and how do you prevent it?

▸ Many requests hit a cold cache simultaneously after expiry and all go to DB. Prevention: mutex lock on cache rebuild, probabilistic early expiry, staggered TTLs.

04 What is a hot key problem in Redis and how do you solve it?

▸ One key gets 90% of traffic and overloads a single Redis node. Solution: local in-memory cache for hot keys, key sharding, read replicas for Redis.

05 When should you NOT cache something?

▸ Data that changes faster than the TTL, financial balances that must be exact, user-specific data that varies per request, anything where staleness causes security issues.

06 What is the difference between a local cache and a distributed cache?

▸ Local: in-process, fast, inconsistent across instances. Distributed (Redis): shared, consistent, network overhead. Use local for session-local data, distributed for shared state.

07 How do you handle cache consistency when data changes?

▸ Event-driven invalidation (publish an event on write, consumers invalidate), write-through (update cache on every write), versioned keys (new key on change).

08 What is a CDN and how is it different from an application-level cache?

▸ CDN caches at the edge, close to users, for static and cacheable HTTP responses. App cache (Redis) is for data. CDN reduces network latency; Redis reduces DB load.

09 What is cache-aside vs read-through caching?

▸ Cache-aside: application code checks cache, falls back to DB, writes to cache itself. Read-through: the cache layer loads from DB on miss automatically. Read-through simplifies app code.

10 How do you size a cache and decide what eviction policy to use?

▸ Size based on working set, not total data volume. LRU for general purpose, LFU when access frequency matters more than recency, TTL-based for time-sensitive data.

TOPIC 05

VERY COMMON

Idempotency

Idempotency is a senior-level topic that most junior devs have never heard of and most mid-level devs know the word but can't implement. It comes up in every payments, messaging, and distributed systems interview.

01 What is idempotency? Why does it matter for APIs?

▸ Calling an operation N times has the same effect as calling it once. Matters because networks are unreliable: clients retry. Without idempotency, retries cause duplicate charges.

02 Which HTTP methods are idempotent by spec? Which aren't?

▸ GET, PUT, DELETE, HEAD are idempotent. POST is not. PATCH is not by spec (though it can be designed to be). GET = read only, PUT = replace, not append.

03 How do you implement idempotency for a payment API endpoint?

▸ Client sends an idempotency key (UUID) in the header. Server stores key + result. On a duplicate request with the same key: return stored result, don't re-execute.

04 How do you store and look up idempotency keys efficiently?

▸ Redis with TTL for fast lookup and automatic expiry. Key: idempotency key. Value: request hash + response. Lock during processing to prevent parallel duplicate execution.

05 What happens if the server processes the request but crashes before responding?

▸ Client retries with the same idempotency key. Server finds the key in storage (if written before crashing) or re-processes. This is why you write the key before processing.

06 How do you handle idempotency in a message queue consumer?

▸ Messages can be delivered more than once. Consumer should check if the message ID was already processed. If yes, ack and skip. If no, process then store.

07 What is the difference between idempotency and exactly-once delivery?

▸ Idempotency is a property of the operation (safe to repeat). Exactly-once delivery is a guarantee from the message system. Idempotency is cheaper and more practical.

08 Can a non-idempotent operation be made safe to retry? How?

▸ Yes: wrap it in a check-then-act pattern with an idempotency key. Check if already executed. If not, execute and store the result. If yes, return the stored result.

09 How would you design idempotency for a bulk/batch endpoint?

▸ Each item in the batch needs its own idempotency key, not just one for the whole batch — otherwise a partial failure can't be safely retried. Track per-item status.

10 Why is idempotency key expiry a security and cost consideration?

▸ If keys never expire, storage grows unbounded. If they expire too fast, a slow retry could replay and duplicate. Balance TTL against a realistic client retry window.

TOPIC 06

VERY COMMON

Message Queues

Message queue questions test if you understand async architecture, not just whether you've used a tool. Interviewers want to know when you'd use a queue and what breaks without one.

01 What is the difference between a message queue and pub/sub?

▸ Queue: one producer, one consumer, message deleted after consumption. Pub/sub: one producer, many subscribers, each gets a copy. Kafka blurs the line by retaining messages.

02 Explain at-most-once, at-least-once, and exactly-once delivery.

▸ At-most-once: fire and forget, may lose messages. At-least-once: retry until acked, may duplicate. Exactly-once: requires deduplication on both sides.

03 What is a dead letter queue and when does a message end up there?

▸ A separate queue for messages that failed after max retries. Prevents a poison pill message from blocking the whole queue indefinitely.

04 What is backpressure and how do message queues help?

▸ Consumer can't keep up with producer. Without a queue: producer overwhelms consumer, system crashes. With a queue: messages buffer, consumer processes at its own pace.

05 How do you handle duplicate messages in a consumer?

▸ Every message must have a unique ID. Consumer checks if it was already processed (DB or Redis). If yes, ack and skip. If no, process then store the ID.

06 What is a consumer group in Kafka and how does it enable scaling?

▸ Multiple consumers in a group share partitions of a topic. Each partition is consumed by exactly one consumer in the group. Max parallelism = number of partitions.

07 When would you use a message queue instead of a direct API call?

▸ When the operation is slow, when you need retry logic, when producer and consumer should scale independently, when you need guaranteed delivery.

08 What is the difference between Kafka and RabbitMQ?

▸ Kafka: log-based, messages retained, high throughput, ordered per partition. RabbitMQ: queue-based, messages deleted after ack, push-based, flexible routing.

09 What is message ordering and when does it break down?

▸ Kafka guarantees order only within a partition, not across partitions. If order matters, related messages need the same partition key.

10 How do you handle a 'poison pill' message that keeps crashing your consumer?

▸ Cap retries, then route to a dead letter queue instead of retrying forever. Alert on DLQ growth. Never let one bad message block the whole queue.

TOPIC 07

VERY COMMON

Distributed Systems

Distributed systems questions separate developers who've built on one server from engineers who've thought about what happens when you have many. Product-company interviews include at least two of these.

01 What is the CAP theorem? Give a real example of a CP and an AP system.

▸ Can only guarantee 2 of 3: Consistency, Availability, Partition tolerance. CP: HBase, Zookeeper. AP: DynamoDB, Cassandra (serve stale data during partition).

02 What is eventual consistency? Give a real-world example of when it's acceptable.

▸ All nodes converge to the same value eventually, not immediately. Acceptable: likes, view counts. Not acceptable: bank balance, inventory during checkout.

03 What is consistent hashing and why is it used?

▸ Maps keys to nodes in a ring. Adding or removing a node only remaps a fraction of keys instead of all keys. Used in distributed caches and load balancers.

04 What is a split-brain problem?

▸ Network partition splits nodes into two groups, each thinks it's the leader. Both accept writes. Solution: quorum (majority must agree before accepting writes).

05 What is a distributed lock and what are its failure modes?

▸ A lock across multiple service instances (Redis SETNX, Redlock). Failure modes: lock holder dies and holds forever (use TTL), clock drift between nodes.

06 What is the two-generals problem and why does it matter?

▸ Two armies can't reliably agree to attack over an unreliable channel. Proof that guaranteed exactly-once delivery over unreliable networks is impossible.

07 What is a heartbeat in distributed systems?

▸ Regular message a node sends to prove it's alive. If missed for N seconds, the node is considered dead and traffic is rerouted. Foundation of failure detection.

08 What is the difference between a service mesh and a load balancer?

▸ Load balancer: distributes traffic, sits in front of services. Service mesh: handles service-to-service communication via sidecar proxies. Complements, doesn't replace.

09 What is a quorum and why does it matter for writes/reads?

▸ Majority of nodes (N/2 + 1) must agree before a write/read is considered successful. Prevents split-brain and stale reads in Raft-based systems.

10 What is the difference between leader-based and leaderless replication?

▸ Leader-based: one node accepts writes, simpler consistency, single point of failure until failover. Leaderless: any node accepts writes, higher availability, needs conflict resolution.

TOPIC 08

ASKED EVERYWHERE

API Design

API design is asked in almost every backend interview because it reveals how you think about contracts, versioning, and the developers who will consume what you build.

01 What makes an API RESTful? What are the constraints?

▸ Stateless, client-server, cacheable, uniform interface, layered system, optional code-on-demand. Stateless is the most important — server holds no client session state.

02 When would you use REST vs GraphQL vs gRPC?

▸ REST: standard web APIs, public APIs. GraphQL: flexible querying, mobile clients. gRPC: internal service-to-service, high throughput, bi-directional streaming.

03 How do you version an API without breaking existing clients?

▸ URL versioning (/v1/, /v2/) is most explicit. Header versioning is RESTful but less discoverable. Never remove/rename fields in a version, only add.

04 What is cursor-based pagination and why is it better than offset?

▸ Offset: skip N rows — gets slower as N grows. Cursor: encode the last seen row's sort key, fetch rows after it. Stable even as data changes.

05 What HTTP status codes should your API return for common scenarios?

▸ 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500 — know when each applies, not just what they mean.

06 How do you design an API endpoint that might take 30 seconds to complete?

▸ Async pattern: POST returns 202 Accepted with a job ID immediately. Client polls GET /jobs/:id for status, or use webhooks.

07 What is HATEOAS and is it worth implementing?

▸ Hypermedia As The Engine Of Application State: responses include links to next valid actions. Technically RESTful, but rarely implemented in practice — high overhead.

08 How do you design backward-compatible API changes?

▸ Add fields, never remove. Make new fields optional. Never change the type of an existing field. Deprecate with a header before removing, provide migration guides.

09 How do you design idempotent POST endpoints for resource creation?

▸ Accept a client-generated idempotency key or unique ID for the resource. A duplicate request with the same key returns the original resource instead of creating a second one.

10 What is rate limiting and how do you communicate it to API consumers?

▸ Limit requests per client per time window (token bucket, sliding window). Communicate via 429 status and Retry-After / rate-limit headers.

TOPIC 09

VERY COMMON

Database Design

Database design questions reveal whether you've designed a schema that worked in production or just one that worked in development. The N+1 problem, migrations, and sharding are the most common deep dives.

01 What is database normalisation? What are 1NF, 2NF, 3NF?

▸ 1NF: atomic values, no repeating groups. 2NF: 1NF + non-key columns depend on the whole primary key. 3NF: 2NF + non-key columns depend only on the primary key.

02 When would you denormalise a database and what are the trade-offs?

▸ When read performance is critical and you can tolerate data duplication. Trade-off: faster reads, slower writes, risk of inconsistency.

03 What is the N+1 query problem? How do you detect and fix it?

▸ Fetching N users then making 1 query per user for their posts = N+1 queries. Fix: JOIN or eager loading. Detect: log queries, look for repeated queries with different IDs.

04 When would you choose SQL over NoSQL and vice versa?

▸ SQL: complex relationships, joins, ACID needed, stable schema. NoSQL: massive scale, flexible schema, simple access patterns. It's about access patterns, not size.

05 What is database sharding? What problems does it introduce?

▸ Splitting data across multiple DB instances by a shard key. Problems: cross-shard queries are hard, transactions across shards need 2PC, hot shards, resharding is painful.

06 How do you run a database migration with zero downtime?

▸ Expand-contract pattern: add new column (nullable), dual-write to old + new, backfill old rows, read from new column, remove old column.

07 What is connection pooling and what happens without it?

▸ Reuses existing DB connections instead of creating a new one per request. Without it: overhead per request, DB hits max connections under load.

08 What is a foreign key constraint and when would you skip it?

▸ Enforces referential integrity at the DB level. Skip when: high write throughput, sharded databases (cross-shard FK impossible), eventual consistency is acceptable.

09 How do you model a many-to-many relationship?

▸ A join/junction table with foreign keys to both tables, plus its own primary key if the relationship has attributes. Tests if you avoid comma-separated ID columns.

10 What is a soft delete and when is it better than a hard delete?

▸ Mark a row deleted (deleted_at) instead of removing it. Better for audit trails, undo, referential integrity. Costs: every query must filter deleted rows.

TOPIC 10

VERY COMMON

Authentication & Authorization

Auth questions test whether you understand identity and permission as two separate problems, and whether you've actually implemented a secure login flow rather than just called a library.

01 What is the difference between authentication and authorization?

▸ Authentication: who are you (verifying identity). Authorization: what are you allowed to do (permissions). Interviewers check candidates don't conflate the two.

02 How does JWT-based authentication work end to end?

▸ Server issues a signed token with claims (user ID, expiry) after login. Client sends it on each request. Server verifies signature and expiry — no DB lookup needed per request.

03 What is the difference between a JWT access token and a refresh token?

▸ Access token: short-lived, used on every request, hard to revoke. Refresh token: long-lived, stored securely, used only to get a new access token.

04 How do you revoke a JWT before it expires?

▸ JWTs are stateless by design, so true revocation needs a blocklist (Redis set of revoked IDs checked per request) or short expiry + refresh token rotation.

05 What is OAuth 2.0 and what problem does it solve?

▸ A delegation protocol: lets a user grant a third-party app limited access to their data on another service without sharing their password.

06 What is the difference between session-based auth and token-based auth?

▸ Session: server stores state, client holds a session ID cookie. Token: server stores nothing, client holds a signed token with all claims. Sessions revoke easily; tokens scale better.

07 What is RBAC vs ABAC?

▸ RBAC: permissions tied to roles (admin, editor, viewer) — simple, coarse-grained. ABAC: permissions computed from attributes — flexible, more complex to reason about.

08 How do you securely store passwords?

▸ Never plaintext or reversible encryption. Hash with a slow, salted algorithm (bcrypt, argon2, scrypt) — never MD5/SHA1 alone.

09 What is CSRF and how do you prevent it?

▸ A malicious site tricks a logged-in user's browser into making a request using their cookies. Prevent with CSRF tokens, SameSite cookies, checking Origin/Referer headers.

10 How do you design authorization checks that scale across microservices?

▸ Centralize policy decisions (a shared auth service or embedded policy library) rather than duplicating logic per service. Pass verified claims via a signed token.

TOPIC 11

COMMON

Docker & Deployment

Deployment questions test whether you understand what actually happens between 'it works on my machine' and 'it's running in production' — containers, rollout strategy, and failure recovery.

01 What is the difference between a Docker image and a container?

▸ Image: a static, immutable blueprint. Container: a running instance of an image, with its own writable layer. One image, many containers.

02 Why do we use multi-stage builds in a Dockerfile?

▸ Build/compile in one stage with dev dependencies, then copy only the final artifact into a clean runtime stage. Reduces image size and attack surface.

03 What is the difference between COPY and ADD in a Dockerfile?

▸ COPY: simple, predictable file copy. ADD: also handles remote URLs and auto-extracts tar files. Prefer COPY unless you specifically need ADD's extra behavior.

04 How do you keep Docker images small and why does it matter?

▸ Minimal base images, multi-stage builds, combined RUN commands, a .dockerignore. Matters for faster deploys, smaller attack surface, faster cold starts.

05 What is a Docker layer and why does layer order matter?

▸ Each instruction creates a cached layer. Put rarely-changing steps before frequently-changing ones so Docker can reuse cached layers and speed up rebuilds.

06 What is the difference between blue-green and rolling deployment?

▸ Blue-green: two full environments, switch traffic all at once, instant rollback. Rolling: gradually replace old instances, slower rollback, no duplicate environment.

07 What is a health check and why does an orchestrator need one?

▸ An endpoint the orchestrator polls to know if a container is ready (readiness) or still alive (liveness). Without it, traffic routes to a crashed container.

08 How do you handle secrets in a containerized deployment?

▸ Never bake secrets into the image or repo. Use environment variables injected at runtime, or a secrets manager (Vault, AWS Secrets Manager, Kubernetes Secrets).

09 What happens during a container restart, and how should your app handle it?

▸ Fresh filesystem, in-memory state and local files are lost. App should be stateless, persist data externally, and handle SIGTERM gracefully for cleanup.

10 What is the difference between horizontal and vertical scaling?

▸ Vertical: bigger machine, simple but has a ceiling and downtime to resize. Horizontal: more instances behind a load balancer, requires stateless design.

TOPIC 12

COMMON

Error Handling & Logging

This is where interviewers check maturity — anyone can write a happy-path endpoint. This topic tests whether you've thought about what happens when things go wrong, and whether you can actually find out why afterward.

01 What is the difference between an operational error and a programmer error?

▸ Operational: expected failure in a working system (timeout, invalid input) — handle gracefully. Programmer error: a bug — usually should crash/restart, not be silently caught.

02 Where should you catch errors in a layered backend application?

▸ Catch where you have enough context to decide (retry, fallback, message). Let unexpected errors bubble up to a top-level handler that logs and alerts.

03 What makes a log line actually useful in production?

▸ Structured (JSON), includes a request/trace ID, timestamp, severity, and context — so you can filter and correlate across a distributed system.

04 What is the difference between log levels and how do you decide what goes where?

▸ DEBUG: verbose dev detail. INFO: normal milestones. WARN: unexpected but recoverable. ERROR: failed and needs attention. Checks you're not logging everything as ERROR.

05 How do you avoid leaking sensitive data in logs or error messages?

▸ Never log raw passwords, tokens, or full PII. Redact sensitive fields before logging. Return generic errors to clients — detail stays in internal logs only.

06 What is a circuit breaker and how does it relate to error handling?

▸ Stops calling a failing downstream service after repeated failures, failing fast instead of piling up timeouts. Prevents cascading failures.

07 How should an API communicate errors to clients in a consistent way?

▸ A consistent error response shape (code, message, details) paired with the correct HTTP status. Avoid returning 200 OK with an error buried in the body.

08 What is the retry-with-backoff pattern and when should you NOT use it?

▸ Retry with increasing delay (exponential backoff, jitter) to avoid hammering a struggling service. Don't use it for non-idempotent operations without a key.

09 How do you design error handling for a batch job processing thousands of records?

▸ Don't let one bad record fail the whole batch. Catch per-record errors, log with context, continue, report a summary at the end.

10 What should happen on an unhandled exception, like an uncaught promise rejection?

▸ Log it with full context immediately, then let the process crash and restart via a process manager. Swallowing it silently is worse than crashing.

TOPIC 13

ASKED EVERYWHERE

Production Debugging

This is the most practical category — interviewers ask scenario questions to see if you've actually been paged at 2am and solved a real problem. Structure your answers: observe, hypothesise, test, fix, prevent.

01 Your API response times doubled overnight. Walk me through debugging this.

▸ Check: recent deployments, DB slow query log, CPU/memory/connections, external API calls, cache hit rate. Narrow to the actual cause systematically.

02 Your database CPU is at 100%. What are the first 5 things you check?

▸ Running queries, slow query log, EXPLAIN on top queries, missing indexes on recent queries, recent schema changes — then lock contention.

03 How do you find a memory leak in a production Node.js service?

▸ Take heap snapshots at intervals, compare growing object counts. Common causes: event listeners not removed, closures holding references, caches without eviction.

04 What is distributed tracing and how does it help debug microservices?

▸ Attaches a trace ID to a request that propagates through every service. You see the full request journey — which service was slow, where the error originated.

05 What is the difference between a metric, a log, and a trace?

▸ Metric: numeric measurement over time. Log: discrete event with context. Trace: end-to-end journey of one request across services.

06 Your error rate is 1% but you can't reproduce the error. How do you debug it?

▸ Filter error logs by type, find a failed request ID, trace what user/input/code path was involved. Add more logging to the suspected path.

07 How do you debug a race condition that only happens in production?

▸ Add structured logs with timestamps and request IDs at both write points. Analyse for overlapping windows. Add a lock. Load test to reproduce at higher concurrency.

08 What should a good post-mortem contain?

▸ Timeline, impact, root cause (not 'human error' — the systemic gap), contributing factors, action items with owners and dates, blameless tone throughout.

09 A customer reports data that 'disappeared' from your system. How do you investigate?

▸ Check if it's a soft delete, a filter bug hiding it, a failed write that never persisted, or a race condition that overwrote it. Check audit logs before assuming loss.

10 How do you debug a service that's healthy by every metric but users report it's 'slow'?

▸ Check client-side timing, not just server metrics — the bottleneck might be outside your service. Check p99/p999 latency, not just averages.

MOST CANDIDATES KNOW THE ANSWERS. FEW KNOW THE REASONING.

An interviewer who asks "what is a deadlock?" isn't looking for a definition. They're looking for whether you've seen one in production, whether you can explain why it happened, and whether you know how to prevent the next one. The definition is the floor. The reasoning is what gets you hired.

HOW TO STUDY THIS

- **Don't memorise answers** — understand the concept until you can explain it to someone who's never heard of it. Then the answer comes naturally.
- **Answer the hint, not just the question** — for each question, read the "what they're testing" note first. That tells you what the interviewer actually wants to hear.
- **Use the situation + trade-off format** — for every answer: what it is, why it exists, what problem it solves, and what it trades off.
- **Prioritise the 5-dot topics** — Database Indexing, Caching, API Design, Transactions, and Production Debugging appear in almost every backend interview. Master those first.
- **Simulate production scenarios** — for debugging questions, don't just answer. Tell a story: "I'd first check X because Y, then if that's clear I'd look at Z." Structure shows seniority.

FINAL THOUGHT

The engineers who do well in backend interviews aren't the ones who've studied the most. They're the ones who've built things, broken things, fixed things, and can talk about the experience honestly. Study this guide. Then go build something that uses every concept in it.